

Shuri's Last Attempt

MIRAGE 2.0 — Reverse Engineering — Official Writeup

CATEGORY	DIFFICULTY	PORT
Reverse Engineering	Hard	1710/tcp
FILES	VULNERABILITY	STATUS
shuri_last_attempt (stripped, PIE)	N/A — custom bytecode VM reversal	Solved — deployment bug found and fixed

Challenge Premise

“Wakanda’s final recovery protocol has been locked behind a damaged Mind Stone interface... the core logic is buried behind its own execution layer.” The menu offers: [1] Request diagnostic trace, [2] Submit recovery sequence, [3] Abort connection. Option [1] is a decoy — it prints an unrelated 64-byte “microcode trace” blob that is never actually decoded or used anywhere.

Reversing the Binary

Option [2] feeds the input into a small custom bytecode interpreter: a fetch-decode-execute loop reading opcodes from an embedded program blob and dispatching through a jump table. The reconstructed instruction set has seven opcodes:

```
0x10  load next key character
0x21  XOR accumulator with an embedded constant
0x22  ADD an embedded constant (mod 256)
0x31  compare accumulator, roll it into a running checksum
0x41  no-op
0x70  finalize / compare the running checksum
0x7f  halt
```

The submitted sequence must be exactly **19 characters**; each character feeds one $0x10 \rightarrow 0x21 \rightarrow 0x22 \rightarrow 0x31$ block, and a final $0x70$ validates a running checksum accumulated across all 19 positions. As with other challenges in this series, two anti-analysis gates guard the whole check:

- `/proc/self/status` TracerPid check.
- A `CLOCK_MONOTONIC_RAW`-bracketed ~120,000-iteration LCG spin, rejecting anything slower than a 25ms budget.

Either one diverts to a decoy branch that prints a plausible but fake flag:

```
MIRAGE{th3_v4ult_b3l0ngs_t0_th4n0s}  <-- decoy, only shown if traced
```

The per-position XOR/ADD constants and the final checksum target were extracted from the decoded bytecode and inverted algebraically in Python (no brute force needed — each position’s arithmetic is independently solvable), recovering the correct 19-character recovery sequence.

Participant Solve Steps

```
$ nc <host> 1710
SHURI@CORE> 2
[INPUT] Recovery sequence: SHURI-PRIME-AXIOM-7
```

FLAG

MIRAGE{shur1_f1n4l_4tt3mpt_br0ke_th3_vm!}

FIX APPLIED: Deployment bug found: the shipped Dockerfile builds FROM `debian:bookworm-slim` (glibc 2.36), but the binary calls `__isoc23_strtol`, which requires **glibc $\geq$ 2.38** — the same packaging bug found in The Final Rip, suggesting both were built with the same newer toolchain. As shipped, every connection attempt crashes before printing anything.

Fix applied to challenge/Dockerfile: changed the base image to `debian:trixie-slim` (glibc 2.41). Rebuilt and confirmed the service now starts correctly and responds on port 1710.

Verification

Rebuilt the fixed image, ran it as a live TCP service, and connected over a plain socket exactly as a participant would; the recovery sequence above produced “[+] FINAL SIGNAL ACCEPTED” followed by the flag shown. Feeding an incorrect sequence of the right length instead produces “[!] Sequence rejected: incorrect length.” only when the length itself is wrong, confirming the 19-character requirement independently.